Adversarially Robust Neural Architectures

Minjing Dong, Yanxi Li, Yunhe Wang, *Senior Member, IEEE*, and Chang Xu, *Senior Member, IEEE*

***Abstract*—Deep Neural Networks (DNNs) are vulnerable to adversarial attacks. Existing methods are devoted to developing various robust training strategies or regularizations to update the weights of the neural network. But beyond the weights, the overall structure and information flow in the network are explicitly determined by the neural architecture, which remains unexplored. Thus, this paper aims to improve the adversarial robustness of the network from the architectural perspective. We explore the relationship among adversarial robustness, Lipschitz constant, and architecture parameters and show that an appropriate constraint on architecture parameters could reduce the Lipschitz constant to further improve the robustness. The importance of architecture parameters could vary from operation to operation or connection to connection. We approximate the Lipschitz constant of the entire network through a univariate log-normal distribution, whose mean and variance are related to architecture parameters. The confidence can be fulfilled through formulating a constraint on the distribution parameters based on the cumulative function. Compared with adversarially trained neural architectures searched by various NAS algorithms as well as efficient human-designed models, our algorithm empirically achieves the best performance among all the models under various attacks on different datasets.**

***Index Terms*—Adversarial robustness, neural architecture search.**

I. Introduction

DEEP neural networks have shown remarkable performance in various applications, such as image classification [1], [2], [3], object detection [4], and machine translation [5], [6]. However, recent works [7], [8], [9], [10], [11] have shown that DNNs are vulnerable to adversarial samples that can fool the networks to make wrong predictions with only perturbations of the input data, which has caused security issues. To deal with the threat of adversarial samples, the majority of existing works focus on robust training which optimizes the weights of robust DNNs through feeding adversarial samples generated by attack approaches (e.g. FGSM). Although the trained networks show good robustness on various attacks, the architectures of these networks are fixed during optimization, which limits the adversarial robustness improvement. The efficient architectures designed by human experts, such as AlexNet and ResNet [3], [12], suggest that the DNN performance is subject to the architecture of network. Recent boosting NAS studies also emphasize the influence of architecture. Hence, we ask a simple question:

Can the network be initialized with robust architecture to further obtain adversarial robustness?

A recent study has shown that different architectures tend to have different levels of adversarial robustness [13]. Thus, the designing of robust neural architectures becomes essential for robustness improvement. However, the problem remains since designing a robust neural architecture can be rather expensive due to the substantial time cost and human effort, and the direct relationship between adversarial robustness and architectures is still unexplored.

To reduce the cost of discovering superior robust neural architecture, we made use of NAS algorithms which automatically discover the ideal architectures within a predefined search space. Recently, remarkable progress has been achieved in NAS, including RL-based approaches [14], [15], [16] and gradient-based approaches [17], [18], [19], [20]. In particular, DARTS [17] introduced a differentiable method for architecture optimization through a continuous relaxation on discrete search space through forming a weighted sum of operations instead of discrete architecture selection, which significantly reduced the searching budget.

Although the NAS framework provided an efficient way to automatically discover superior neural architectures with customized objectives, the standard adversarial training required massive costs to generate adversarial examples, which significantly decreased search efficiency. Thus, we tried to dismiss the inner maximum of adversarial training to further accelerate the optimization of architecture through involving the Lipschitz constraint by exploring the influence of Lipschitz constant on adversarial robustness and how the architecture parameters impact the Lipschitz constant. In this paper, we proposed to explore the relationship between adversarial robustness and the architecture of network through establishing their connections to Lipschitz constant under NAS framework.

Furthermore, the instability of differentiable NAS algorithms has been explored by previous work [21]. Existing differentiable NAS algorithms utilize architecture parameters for sampling superior architectures, where all the elements of architecture parameters are "equally treated" for selection without exploring their discrepancies. For example, two nodes in the same cell may have different levels of freedom for operation selection, however, in existing algorithms, they were only assigned with

Received 31 December 2022; revised 29 December 2024; accepted 5 February 2025. Date of publication 17 February 2025; date of current version 3 April 2025. This work was supported in part by the Start-up of the City University of Hong Kong under Grant 9610680, in part by Young Scientist Fund of NSFC under Grant 62406265, and in part by the Australian Research Council under Projects under Grant DP240101848 and Grant FT230100549. Recommended for acceptance by H. Su. *(Corresponding author: Chang Xu.)*

Minjing Dong is with the Department of Computer Science, City University of Hong Kong, Kowloon, Hong Kong (e-mail: minjdong@cityu.edu.hk).

Yanxi Li and Chang Xu are with the School of Computer Science, Faculty of Engineering, University of Sydney, Darlington, NSW 2008, Australia (e-mail: yali0722@unisydney.edu.au; c.xu@sydney.edu.au).

Yunhe Wang is with the Noah's Ark Laboratory, Huawei Technologies Company, Ltd, Beijing 100085, China (e-mail: yunhe.wang@huawei.com).

Digital Object Identifier 10.1109/TPAMI.2025.3542350

0162-8828 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

trainable parameters and applied with $argmax$ for selection after searching, which significantly reduced the reliability of sampled architecture and raises a demand for confidence learning of architecture parameters. Thus, we proposed to sample architecture parameters from trainable distributions instead of initializing them directly.

Our proposed algorithm Adversarially Robust Neural Architecture Search with Confidence Learning (RACL) starts from an approximation of Lipschitz constant of entire neural network under NAS framework, where we derive the relationship between Lipschitz constant and architecture parameters. We further propose to sample architecture parameters from log-normal distributions. Utilizing the properties of log-normal distribution, we show that the Lipschitz constant of entire network can be approximated with another log-normal distribution with mean and variance related to architecture parameters so that a constraint can be formulated in the form of a cumulative function to achieve Lipschitz constraint on the architecture. Our algorithm achieves an efficient robust architecture search and RACL empirically achieves superior adversarial robustness compared with other NAS algorithms as well as state-of-the-art models through a series of experiments under different settings.

II. Related Work

To situate the current work, we review some relevant literature that covers adversarial attacks, defense methods, and neural architecture search.

A. Adversarial Attacks

Szegedy et al. first revealed the adversarial samples, which demonstrated that neural networks are vulnerable to adversarial attacks [22]. Given a fixed input, through utilizing the model gradient w.r.t the input, a perturbation that wildly changes the predicted output can be easily found. Vast techniques have been introduced to generate powerful adversarial samples in an efficient way. Adversarial attacks are generally divided into two groups, the white-box case [9], [23], [24], [25] and closed-box case [26], [27], [28], [29]. The white-box cases enable attacks have full access to the network. Goodfellow et al. introduced an efficient attack method FGSM through a one-step gradient-based attack on the input [9]. Kurakin et al. [30] first proposed an iterative attack methods I-FGSM instead of a one-step gradient-based attack to obtain much more powerful attacks. Dong et al. [31] proposed to integrate momentum into the I-FGSM for more stable updating and boost the transferability of generated adversarial samples. Mardry et al. [32] introduced a strong attack, Projected Gradient Descent (PGD), which is now widely used in robustness learning. PGD attack utilized the local first-order information of the network to achieve high attack success rates. Universal adversarial perturbations have also been studied by previous work [33]. On the contrary, the closed-box attacks where the model architecture and parameters are not accessible are relatively weak. However, the closed-box attacks fit the actual circumstances better and thus have received much attention. Madry et al. [32] explored the transferability phenomenon of adversarial attacks which showed that the generated adversarial samples can also achieve relatively high attack success rates on another network. Besides transfer-based closed-box attacks, some query-based attacks were introduced where adversaries can only query the outputs of the models [28], [34]. Yan et al. [35] proposed to bridge the gap between transfer-based and query-based attacks to achieve more efficient closed-box attacks. Besides the attacks on the classification tasks, adversarial attacks have been applied to other tasks, such as detection and segmentation [36], [37].

B. Defense Mechanisms Against Attacks

Due to the exponential growth of attack approaches, more attention have been paid to defense methods recently which tackled the vulnerability of neural networks through improving the adversarial robustness. There are various defense mechanisms proposed by previous work. Gradient Masking methods hided the gradient information to confound the adversaries [38], [39], however, they cannot defend attacks based on approximate gradient [40]. Adversarial Example Detection is another stream which aims at discovering the adversarial examples and rejects them [41], [42]. Feinman et al. [43] proposed to randomize the classifier with Dropout to identify the adversarial examples based on the prediction variance. The main stream of defense Mechanisms is the robust optimization which further optimizes the network to achieve adversarial robustness. Adversarial training is naturally introduced to defend attacks through feeding adversarial examples into the training stage to form a min-max game where the inner maximum generates adversarial samples to maximize the classification loss and outer minimum optimizes model parameters to minimize the loss. Different attack strategies have been applied to generate adversarial examples for adversarial training, such as PGD attack [32] and FGSM attack [23]. Besides the standard adversarial training, different variants have been proposed. Miyato et al. [44] proposed virtual adversarial training which defines the adversarial direction without label information. Shafahi et al. [45] introduced an efficient adversarial training which recycled the gradient information. Zhang et al. [46] proposed to decompose the prediction error into the classification and boundary error and provided a tight upper bound. Pang et al. [47] introduced adaptive diversity promoting (ADP) which improved the adversarial robustness of ensemble models. Some regularization methods have been introduced to defend against attacks. Cisse et al. and Weng et al. [48], [49] proposed to constrain the Lipschitz constant of network to improve the adversarial robustness. Mustafa et al. [50] introduced an effective constraint which forced the features for each class to lie inside a convex polytope and separated from those of other classes.

C. Neural Architecture Search

Although vast approaches have been proposed to defend against adversarial samples, most of them focused on optimizing the weights based on different strategies, and the impact of architecture has been ignored. Recently, neural architecture search has received increasing attention due to its superior performance. Early NAS approaches heavily relied on macro searching which

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

directly searches the entire network [14], [51]. For efficiency, more NAS approaches have applied micro search space where the cell is searched instead of the entire network, and the cells are stacked in series to compose the whole network [52], [53]. Yang et al. [54] proposed an efficient continuous evolutionary approach on the supernet where all the architectures share the parameters, which boosted the searching efficiency. Recently, the differentiable searching algorithm DARTS has been introduced to boost the searching speed through a relaxation on search space to form a supernet with operation mixture to achieve differentiable architecture searching [17]. Dong et al. [18] introduced a differentiable sampler over the supernet with Gumbel-Softmax which improved the searching efficiency. Xu et al. [19] proposed to apply channel sampling for searching acceleration and add edge normalization to stabilize the searching phase. Dong et al. [55] introduced a benchmark of NAS, named NAS-Bench-201, which provides all possible architectures and their trained weights within a pre-defined search space.

D. Robust Neural Architecture Search

Recently, NAS has been applied to different areas, including adversarial robustness. Guo et al. [13] empirically demonstrated that different architectures had different levels of robustness and proposed feature flow guided search to discover the robust neural architectures. Chen et al. [56] proposed ABanditNAS with an improved conventional bandit algorithm to search the architectures under enlarged search space to better defend against adversarial attacks. Kotyan et al. [57] investigated the potential robust architecture in a broader search space including the concatenation and connections between dense and convolution layers, and demonstrated that there exist robust architectures which achieve inherent accuracy on adversarial examples. Mok et al. [58] introduced AdvRush which utilizes the connection between the intrinsic robustness and the smoothness of loss landscape to discover robust architectures. Hosseini et al. [59] proposed DSRNA which searches robust architectures via two differentiable metrics, i.e., certified lower bound and Jacobian norm bound. Jung et al. [60] further studied the connection between architecture designs and adversarial robustness by constructing a NAS benchmark. Devaguptapu et al. [61] empirically explored how the network topology influences adversarial robustness and showed that a simple ensembling can boost robust accuracy. Huang et al. [62] focused on the ResNet block design and introduced RobustResBlock which improves the adversarial robustness. Liu et al. [63] conducted a systematical investigation of the relationship between robust generalization and architectural design of CNNs and ViTs. Different from [57], our objective aims at discovering the robust architecture in the current popular search space benchmark [17], [18] without expensive adversarial training via investigating the connection between Lipschitz constraint and adversarial robustness of architecture.

III. Methodology

In this section, we introduce the proposed robust architecture search with confidence learning (RACL) algorithm. Different from existing defense methods which only focus on weight optimization, we lay emphasis on the influence of architecture on adversarial robustness through exploring the relationship among robustness, architecture, and Lipschitz constant. Confidence learning is further involved, which forms a Lipschitz constraint on architecture parameters. With the proposed algorithm, the searched architectures can have stronger defensive power against adversarial examples.

A. Preliminary

Given the input $x \in \mathbb{R}^D$ and annotated label vector $y \in \mathbb{R}^M$ where M is the total number of classes, the neural network $\mathcal{H}$ maps perturbed input $\tilde{x} = x + \delta$ to a label vector $\hat{y} = \mathcal{H}(\tilde{x}; W, \mathcal{A})$. The network architecture is represented by $\mathcal{A}$, and its filter weight is denoted as W. The objective of adversarial attacks is to find the perturbed input $\tilde{x}$ which leads to wrong predictions through maximizing the classification loss as

$$\tilde{x} = \underset{\tilde{x}:\|\tilde{x}-x\|_p \leqslant \epsilon}{\operatorname{argmax}} \mathcal{L}_{CE}(\mathcal{H}(\tilde{x}; W, \mathcal{A}), y), \tag{1}$$

where $\mathcal{L}_{CE}(\hat{y}, y) = -\sum_{i=1}^{M} y^{(i)} log(\hat{y}^{(i)})$, and the perturbation is constrained by its l_p-norm. Various powerful attacks have been proposed and shown high attack success rates, such as Fast Gradient Sign Method (FGSM) [22] and Projected Gradient Descent (PGD) [32]. To defend against these attacks, regularizing the weight matrix of each layer to form a Lipschitz-constrained network has been proven to be beneficial for the adversarial robustness [48], [49].

Let $\mathcal{F} = \mathcal{L} \circ \mathcal{H}$ be the mapping from the input to the classification loss, and the difference of loss after an adversarial attack can be bounded as

$$\|\mathcal{F}(x + \delta, y; W, \mathcal{A}) - \mathcal{F}(x, y; W, \mathcal{A})\| \leqslant \lambda_{\mathcal{F}} \|\delta\|, \tag{2}$$

where $\lambda_{\mathcal{F}}$ is the Lipschitz constant of function $\mathcal{F}$ with respect to $\|.\|_p$. Together with $\|\delta\|_p \leqslant \epsilon$, the generalization error with perturbed input can be bounded as

$$\begin{aligned}\mathbb{E}_{x \backsim \mathcal{D}}[\mathcal{F}(\tilde{x})] &\leqslant \mathbb{E}_{x \backsim \mathcal{D}}[\mathcal{F}(x)] + \mathbb{E}_{x \backsim \mathcal{D}}[\max_{\|\tilde{x}-x\| \leqslant \epsilon} |\mathcal{F}(\tilde{x}) - \mathcal{F}(x)|] \\ &\leqslant \mathbb{E}_{x \backsim \mathcal{D}}[\mathcal{F}(x)] + \lambda_{\mathcal{F}} \cdot \epsilon,\end{aligned} \tag{3}$$

which suggests that neural networks can defend against adversarial examples with a smaller Lipschitz constant. Although it is difficult to derive the precise Lipschitz constant $\lambda_{\mathcal{F}}$ given a network, we can impose constraints on both the lower bound and upper bound of Lipschitz constant, which are denoted as $\underline{\lambda_{\mathcal{F}}}$ and $\overline{\lambda_{\mathcal{F}}}$ respectively. Thus, an adversarial robust formulation of neural architectures can be written as

$$\min_{\mathcal{A}, W} \mathbb{E}[\mathcal{F}(x, y; W, \mathcal{A})] \; s.t. \; \underline{\lambda_{\mathcal{F}}^*} \leqslant \lambda_{\mathcal{F}} \leqslant \overline{\lambda_{\mathcal{F}}^*}, \tag{4}$$

where $\underline{\lambda_{\mathcal{F}}^*}$ and $\overline{\lambda_{\mathcal{F}}^*}$ are the optimal lower and upper bounds of Lipschitz constant. Existing works often consider a fixed network architecture $\mathcal{A}$ in (4), and focus on optimizing network weight for improved robustness, where the influence of architecture is ignored. Recent studies highlight the importance of architecture. Liu et al. [64] conduct experiments to empirically demonstrate that better trade-offs of some pruning techniques mainly come

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

from the architecture itself. NAS algorithms are introduced to involve architecture optimization to obtain better performance with small model size [17], [19]. We are therefore motivated to investigate the influence of neural architecture on adversarial robustness.

B. Lipschitz Constraints in Neural Architecture

The discrete architecture $\mathcal{A}$ is determined by connections and operations, which creates a huge search space. Differentiable architecture search algorithms provide an efficient solution through the continuous relaxation of the architecture representation [17], [65], [66]. Within the differentiable NAS framework, we decompose the entire neural network into cells. Each cell I is a directed acyclic graph (DAG) consisting of an ordered sequence of n nodes, where each node denotes a latent representation that is transformed from two previous latent representations and each edge (i, j) denotes an operation o from a pre-defined search space $\mathcal{O}$ which transforms $I^{(i)}$. Following [19], the architecture parameters α that weighs operations and β that weighs input flows are introduced to form an operation mixture with weighted inputs. The intermediate node is computed as

$$I^{(j)} = \sum_{i<j} \beta^{(i,j)} \sum_{o\in\mathcal{O}} \alpha_o^{(i,j)} \cdot o(I^{(i)}), \tag{5}$$

where $I^{(0)}$ and $I^{(1)}$ are fixed as input nodes during the searching phase and the last node is formed by channel-wise concatenating of previous intermediate nodes $I = \cup_{i=2}^{n-1} I^{(i)}$ as the cell output. The entire neural network is constructed through two different types of cells including the normal cell, where all the operations have strides of 1, and the reduction cell, where the operations connected to the two inputs have strides of 2. With normal and reduction cells stacked in series, the entire neural network can be formed as $\mathcal{H} = I_1 \circ I_2 \circ \cdots \circ I_N \circ \mathcal{C}$, where N denotes the number of cells and $\mathcal{C}$ denotes the classifier. Following [19], after the searching phase, the operation o with the maximum $\beta^{(i,j)}\alpha_o^{(i,j)}$ for each edge (i, j) is selected and the connection of each node j to its two precedents $i < j$ with maximum $\beta^{(i,j)}\alpha_o^{(i,j)}$ is selected so that a discrete superior architecture can be sampled from the supernet. We now explore the relationship between architecture parameters α, β, and the Lipschitz constant of the network. Since the entire neural network is constructed by stacking cells in series as $[I_1, I_2, \ldots, I_N]$, (2) can be further decomposed as

$$\begin{aligned}\|\mathcal{F}(\tilde{x}) - \mathcal{F}(x)\| &\leqslant \lambda_l \|\mathcal{H}(\tilde{x}) - \mathcal{H}(x)\| \\ &\leqslant \lambda_l \lambda_{\mathcal{C}} \|I_N(\tilde{x}) - I_N(x)\| \\ &\leqslant \lambda_l \lambda_{\mathcal{C}} \lambda(I_N) \|I_{N-1}(\tilde{x}) - I_{N-1}(x)\|,\end{aligned} \tag{6}$$

where λ_l, $\lambda_{\mathcal{C}}$ and $\lambda(I_N)$ denote the Lipschitz constants of the loss function, classifier, and cell I_N respectively. By rewriting $\|I_{N-1}(\tilde{x}) - I_{N-1}(x)\|$ in a format of its previous cells till the input of cell becomes the image for I_1 and considering $\|I_1(\tilde{x}) - I_1(x)\| \leqslant \lambda(I_1)\|\tilde{x} - x\| = \lambda(I_1)\|\delta\|$, (6) can be unfolded recursively and rewritten as

$$\|\mathcal{F}(\tilde{x}) - \mathcal{F}(x)\| \leqslant \lambda_{\mathcal{F}} \|\delta\| \leqslant \|\delta\| \lambda_l \lambda_{\mathcal{C}} \prod_k^N \lambda(I_k). \tag{7}$$

It is obvious that the adversarial robustness can be bounded by the Lipschitz constants of cells. Equation (7) also suggests that the impact of perturbation grows exponentially with the number of cells, which highlights the importance of cell designing. As λ_l and $\lambda_{\mathcal{C}}$ in (7) are not related to the architecture, we next focus on the discussion on $\lambda(I_k)$. Based on the operation mixture defined in (5), the variation of node $I_k^{(j)}$ under perturbation can be written in a format of that in previous node $I_k^{(j)}$. For simplicity of notation, we omit the subscript k and for each node, and we have

$$\begin{aligned}&\|I^{(j)}(\tilde{x}) - I^{(j)}(x)\| \leqslant \sum_{i<j} \beta^{(i,j)} \lambda^{(i,j)} \|I^{(i)}(\tilde{x}) - I^{(i)}(x)\|, \\ &\text{s.t. } \lambda^{(i,j)} \leqslant \sum_{o\in\mathcal{O}} \alpha_o^{(i,j)} \lambda_o,\end{aligned} \tag{8}$$

where $\lambda^{(i,j)}$ denotes the Lipschitz constant of transformation from node i to j and λ_o denotes the Lipschitz constant of operation o. Similarly, we can unfold (8) recursively for the entire cell by rewriting $\|I^{(i)}(\tilde{x}) - I^{(i)}(x)\|$ in a format of its previous node, which can be formulated as

$$\lambda(I^{(j)}) \leqslant \sum_{i<j} \beta^{(i,j)} \sum_{o\in\mathcal{O}} \alpha_o^{(i,j)} \lambda_o. \tag{9}$$

Through substituting $\lambda(I_k)$ in (7) by the one in (9) and taking λ_l and $\lambda_{\mathcal{C}}$ as a unified constant C, the Lipschitz constant $\lambda_{\mathcal{F}}$ is bounded by the product of the Lipschitz constant of intermediate nodes as

$$\begin{aligned}\lambda_{\mathcal{F}} &\leqslant C \prod_k^N \lambda(I_k) \leqslant C \prod_k^N \prod_j^n \lambda(I^{(j)}) \\ &\leqslant C \prod_k^N \prod_j^n \sum_{i<j} \beta^{(i,j)} \sum_{o\in\mathcal{O}} \alpha_o^{(i,j)} \lambda_o.\end{aligned} \tag{10}$$

According to the definition, the Lipschitz constant of operations without convolutional layers can be summarized as follows, (1). average pooling: $S^{-0.5}$ where S denotes the stride of pooling layer, (2). max pooling: 1, (3). identity connection: 1, (4). Zeroize: 0. For the rest operations including depth-wise separate conv and dilated depth-wise separate conv, we focus on the L_2 bounded perturbations and according to the definition of spectral norm, the Lipschitz constant of these operations is the spectral norm of its weight matrix where $\lambda_2^o = \|W^o\|_2$, which also is the maximum singular value of W, marked as Λ_1. However, directly computing Λ_1 is not practical through gradient descent. To achieve a differentiable optimization on the Lipschitz constant, we make use of the power iteration method which can be applied for an efficient approximation of Λ_1 [67]. Note that although the perturbation is L_2 bounded, the robustness against L_∞ can also be achieved, as stated by [68].

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

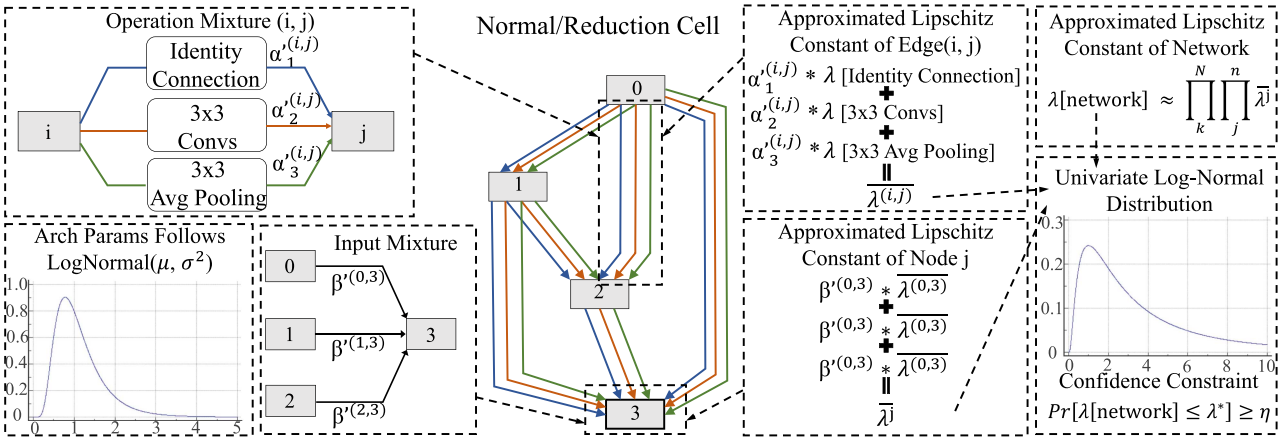


Fig. 1. An overview of proposed robust neural architecture search with confidence learning algorithm. Each node in the cell is computed with operations mixture under architecture parameters α for weighting operations and β for weighting inputs where α and β are sampled from multivariate log-normal distributions. Meanwhile, the Lipschitz constants of each edge and cell are all induce to univariate log-normal distributions. The Lipschitz constraint is formulated from the cumulative distribution function.

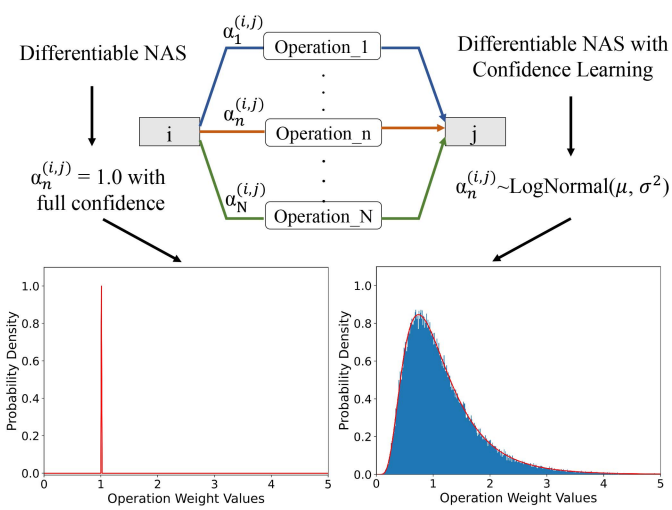


Fig. 2. An illustration of the difference between previous differentiable NAS and ours with confidence learning.

C. Confident Architecture Sampling

The architecture is determined by parameters α and β, which further influence the Lipschitz constants of the network, as shown in (10). Existing NAS algorithms always initialize them as trainable parameters without in-depth analysis. However, these weightings on operations or connections naturally could have different levels of importance and freedom. *E.g.*, the connection of the first node and one of the intermediate nodes may have different levels of freedom for the final selection, however, they are simply assigned with values that share the same confidence for optimization, and then sampled using the maximum value in NAS framework. Thus, previous architecture parameters can hardly fulfill this requirement. Instead, we propose to explore the confidence of the architecture parameters by regarding them as variables sampled from distributions during architecture search. An illustration of the advantage of confidence learning is shown in Fig. 2. For all architecture parameters, previous differentiable NAS algorithms made use of trained value with full confidence as shown in the left part, while our algorithm enables parameters to exploit their confidence as shown in the right part. Intuitively, the architecture optimization is highly uncertain due to the large search space. But existing NAS trusts all architecture parameters without discriminating the confidence on them. Taking α and β from the perspective of distributions, the variances will be optimized to indicate confidence in the values. RACL tends to exploit the operations of higher confidence and explore more potential good paths by investigating operations of lower confidence. The overall searching space can thus be well explored and exploited.

For distributions, a naive selection can be a multivariate normal distribution. However, according to the Lipschitz constant form in (10), the sampled values from this distribution need to be positive since $\lambda_{\mathcal{F}}$ is always positive and negative values from distribution will make the constraint cease to be effective. Thus, we turn to log-normal distribution $\mathcal{LN}$ since it guarantees positive sampled values. Note that a random variable is log-normally distributed $\mathcal{LN}(\mu, \Sigma)$ if the logarithm of it is normally distributed $\mathcal{N}(\mu, \Sigma)$. For simplicity, the following mean and variance denote those of the logarithm. Most importantly, there are several nice properties, including the weighted sum of multiple independent $\mathcal{LN}_{1,\ldots,n}$ can be approximated with another $\mathcal{LN}$ and the product of multiple independent $\mathcal{LN}_{1,\ldots,n}$ induces to $\mathcal{LN}$ with parameters μ and Σ of the sum of those in $\mathcal{LN}_{1,\ldots,n}$. Thus we propose to sample α from multivariate log-normal distributions, denoted as $\mathcal{LN}(\mu^\alpha, \Sigma^\alpha)$, with mean $\mu^\alpha \in \mathbb{R}^d$ and covariance matrix $\Sigma^\alpha \in \mathbb{R}^{d\times d}$ with diagonal standard deviation $\sigma^\alpha \in \mathbb{R}^d$ where d denotes the dimension of α. Similarly, we sample β from $\mathcal{LN}(\mu^\beta, \Sigma^\beta)$.

Back to the Lipschitz constant, the multivariate log-normal distribution over α induces a univariate log-normal distribution over the upper boundary of Lipschitz constant of edge based on the operation mixture $\sum_{o\in\mathcal{O}} \alpha_o^{(i,j)}\lambda_o$ since it can be treated as the weighted sum of multiple log-normal distributed variables. Note that λ_o can be treated as constant since the weights are fixed when optimizing architecture parameters. The proposed Lipschitz confidence constraint is shown in Fig. 1. Although there

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

is no closed-form expression of its probability density function, the distribution can be approximated using the properties below:

Property 1: If a log-normal variable $X \backsim \mathcal{LN}(\mu, \sigma^2)$ is multiplied by a constant a, $aX \backsim \mathcal{LN}(\mu + ln(a), \sigma^2)$.

Property 2: If multiple independent log-normal variables, denoted as $X_1, X_2, \ldots, X_n$, are multiplied, $X_1 \cdot X_2 \cdots X_n \backsim \mathcal{LN}(\sum_{i=1}^n \mu_i, \sum_{i=1}^n \sigma_i^2)$

Following [69], the sum of log-normal distributions can be approximated by another log-normal distribution as below:

Proposition 1: If multiple independent log-normal variables, denoted as $X_1, X_2, \ldots, X_n$, are added, the sum $Z = \sum_{i=1}^n X_i$ can be approximated by another log-normal distribution $\mathcal{LN}(\mu_Z, \sigma_Z^2)$ with variance $\sigma_Z^2 = ln[\frac{\sum e^{(2(\mu_i)+\sigma_i^2)}(e^{(\sigma_i)^2}-1)}{(\sum e^{(\mu_i+\sigma_i^2/2)})^2} + 1])$ and mean $\mu_Z = ln[\sum e^{(\mu_i+\sigma_i^2/2)}] - \frac{\sigma_Z}{2}$.

Thus, in (8), the distribution of $\sum_{o\in\mathcal{O}} \alpha_o^{(i,j)}\lambda_o$ can be treated as a weighted sum of multiple independent log-normal distributions and can be approximated with these properties and Proposition 1. Similarly, in (9), $\sum_{i<j} \beta^{(i,j)} \sum_{o\in\mathcal{O}} \alpha_o^{(i,j)}\lambda_o$ can be treated as the sum of multiple products of two independent log-normal distributions which can also be approximated. Based on the properties of log-normal distribution, the upper boundary of Lipschitz constant of entire network can be approximated accordingly,

$$\begin{aligned}
\overline{\lambda^{(i,j)}} &= \sum_{o\in\mathcal{O}} \alpha_o^{(i,j)}\lambda_o \\
&\backsim \mathcal{LN}\left(ln\left[\sum_o e^{(\mu_o^{\alpha\prime}+(\sigma_o^\alpha)^2/2)}\right] - \frac{\sigma_{I^{(i,j)}}^2}{2}, \sigma_{I^{(i,j)}}^2\right), \\
\sigma_{I^{(i,j)}}^2 &= ln\left[\frac{\sum_o e^{(2(\mu_o^{\alpha\prime})+(\sigma_o^\alpha)^2)}(e^{(\sigma_o^\alpha)^2}-1)}{(\sum_o e^{(\mu_o^{\alpha\prime})+(\sigma_o^\alpha)^2/2)})^2} + 1\right]\Bigg), \\
\mu_o^{\alpha\prime} &= \mu_o^\alpha + ln(\lambda_o)
\end{aligned} \tag{11}$$

where $\overline{\lambda^{(i,j)}}$ denotes the upper boundary of $\lambda^{(i,j)}$. For simplicity, we denote the mean for $\overline{\lambda^{(i,j)}}$ as $\mu_{I^{(i,j)}}$ and variance as $\sigma_{I^{(i,j)}}^2$. Similarly, we sample β from a multivariate log-normal distribution $\mathcal{N}(\mu^\beta, \Sigma^\beta)$. For variable $\beta^{(i,j)}\overline{\lambda^{(i,j)}}$, it can be treated as the product of two log-normal distributions, which also follows a log-normal distribution whose mean is the sum of means of two distributions and variance as well. Thus, to generalize the distribution over edge $\overline{\lambda^{(i,j)}}$ to the one over intermediate node $\overline{\lambda^{(j)}}$, we replace o with j, $\mu_o^\alpha + ln(\lambda_o)$ with $\mu_{(i,j)}^\beta + \mu_{I^{(i,j)}}$, and $(\sigma_o^\alpha)^2$ with $(\sigma_{(i,j)}^\beta)^2 + \sigma_{I^{(i,j)}}^2$ in (11), which obtains the distribution parameters of $\overline{\lambda^{(j)}} = \sum_{i<j} \beta^{(i,j)}\overline{\lambda^{(i,j)}}$ as

$$\begin{aligned}
\overline{\lambda^{(j)}} &= \sum_{i<j} \beta^{(i,j)}\overline{\lambda^{(i,j)}} \\
&\backsim \mathcal{LN}\left(ln\left[\sum_o e^{(\mu_{(i,j)}^{\beta\prime}+[\sigma_{(i,j)}^{\beta\prime}]/2)}\right] - \frac{\sigma_{I^{(j)}}^2}{2}, \sigma_{I^{(j)}}^2\right),
\end{aligned}$$

where $\mu_{(i,j)}^{\beta\prime} = \mu_{(i,j)}^\beta + \mu_{I^{(i,j)}}$,

$$\sigma_{(i,j)}^{\beta\prime} = (\sigma_{(i,j)}^\beta)^2 + \sigma_{I^{(i,j)}}^2,$$

$$\sigma_{I^{(j)}}^2 = ln\left[\frac{\sum_j e^{(2(\mu_{(i,j)}^{\beta\prime})+[\sigma_{(i,j)}^{\beta\prime}])}(e^{[\sigma_{(i,j)}^{\beta\prime}]}-1)}{(\sum_o e^{(\mu_{(i,j)}^{\beta\prime}+[(\sigma_{(i,j)}^{\beta\prime}]/2)})^2} + 1\right]\Bigg). \tag{12}$$

For simplicity, we denote its mean and variance as $\mu_{I^{(j)}}$ and $\sigma_{I^{(j)}}^2$. According to (10), $\lambda_\mathcal{F}$ is bounded by the product of $\overline{\lambda^{(j)}}$. Thus, $\overline{\lambda_\mathcal{F}}$ follows the log-normal distribution with mean $\mu = In(C) + \sum_k^N \sum_j^n \mu_{I^{(j)}}$ and variance $\sigma^2 = \sum_k^N \sum_j^n \sigma_{I^{(j)}}^2$. We introduce a confidence hyperparameter $\eta \in [0, 1]$ to enable confidence learning with such an constraint as

$$\begin{aligned}
&Pr_{\alpha,\beta}[\overline{\lambda_\mathcal{F}} \leqslant \overline{\lambda_\mathcal{F}^*}] \\
&= Pr_{\alpha,\beta}[C\prod_k^N \prod_j^n \sum_{i<j} \beta^{(i,j)} \sum_{o\in\mathcal{O}} \alpha_o^{(i,j)}\lambda_o \leqslant \overline{\lambda_\mathcal{F}^*}] \geqslant \eta,
\end{aligned} \tag{13}$$

where $\overline{\lambda_\mathcal{F}^*}$ is the desired Lipschitz constant upper bound of $\mathcal{F}$. Note that in (13), the variance of $\overline{\lambda_\mathcal{F}}$ is reduced to satisfy the inequality, which strengths the confidence on the approximation of Lipschitz constant of $\lambda_\mathcal{F}$, compared with the one in (10) without confidence learning. To obtain a convex constraint in μ and Σ, we reformulate (13) through the format of cumulative function as

$$\begin{aligned}
Pr[\overline{\lambda_\mathcal{F}} \leqslant \overline{\lambda_\mathcal{F}^*}] &= Pr[\frac{ln(\overline{\lambda_\mathcal{F}}) - \mu}{\sigma} \leqslant \frac{ln(\overline{\lambda_\mathcal{F}^*}) - \mu}{\sigma}] \\
&= \Phi(\frac{ln(\overline{\lambda_\mathcal{F}^*}) - \mu}{\sigma}),
\end{aligned} \tag{14}$$

where Φ denotes the cumulative function of the normal distribution since $\frac{ln(\overline{\lambda_\mathcal{F}})-\mu}{\sigma}$ is a random variable following the normal distribution. Thus, we establish direct relationship among μ, σ and η as

$$\frac{ln(\overline{\lambda_\mathcal{F}^*}) - \mu}{\sigma} \geqslant \Phi^{-1}(\eta). \tag{15}$$

By omitting the square root on σ, a convex constraint can be achieved. Besides the upper bound of Lipschitz constant, we propose to minimize the lower bound $\underline{\lambda_\mathcal{F}}$ together to better control $\lambda_\mathcal{F}$. Taking the advantage of the fact that $\|\nabla\mathcal{F}(x, y; W, \mathcal{A})\| \leq \lambda_\mathcal{F}$, we simply take $\underline{\lambda_\mathcal{F}} = \|\nabla\mathcal{F}(x, y; W, \mathcal{A})\|$. Together with the constraint in (15), we reformulate the optimization objective in (4) as

$$\begin{aligned}
&\min_{\mu^\alpha,\Sigma^\alpha,\mu^\beta,\Sigma^\beta,W} \mathcal{L}_{CE}(\mathcal{F}(x; W, \mathcal{A}), y) + \|\nabla\mathcal{F}(x, y; W, \mathcal{A})\|, \\
&\text{s.t. } ln(\overline{\lambda_\mathcal{F}^*}) - \mu \geqslant \Phi^{-1}(\eta)\sigma^2, \\
&\text{where } \mathcal{A} \backsim \mathcal{LN}(\mu^\alpha, \Sigma^\alpha), \mathcal{LN}(\mu^\beta, \Sigma^\beta).
\end{aligned} \tag{16}$$

Intuitively, the constraint in (16) reveals the influence of σ on sampling architecture parameters. As σ increases, the value of μ decreases to satisfy the inequality where the corresponding μ^α and μ^β become 0 for relatively large σ, which implies that the operations or connections are unlikely to be sampled when its corresponding confidence is low. Thus, the architecture can be sampled based on its confidence in the Lipschitz constraint. We apply the ADMM optimization framework to solve this constrained optimization by incorporating the constraint to form

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Robust Neural Architecture Search With High Confidence Algorithm.

Input: The training set is split into $\mathcal{D}_T$ and $\mathcal{D}_V$; Batch size n; Hyperparameter λ^*, ρ, η;
Initialize multivariate log-normal distributions $\mathcal{LN}(\mu^\alpha, \Sigma^\alpha)$ and $\mathcal{LN}(\mu^\beta, \Sigma^\beta)$; Initialize $\mathcal{H}$ with W;
while not converge **do**
Sample α and β from $\mathcal{LN}(\mu^\alpha, \Sigma^\alpha)$ and $\mathcal{LN}(\mu^\beta, \Sigma^\beta)$ based on reparameterization trick
Sample batch of data $\{(x_i, y_i)\}_{i=1}^n$ from $\mathcal{D}_T$
Optimize W with $\sum_{i=1}^n \mathcal{L}_{CE}(x_i, y_i; W, \alpha, \beta) + \underline{\lambda_{\mathcal{F}}}$
Sample batch of data $\{(x_i, y_i)\}_{i=1}^n$ from $\mathcal{D}_V$
Optimize $\mu^\alpha, \Sigma^\alpha, \mu^\beta, \Sigma^\beta$ with ADMM framework
$\mu_{t+1} \leftarrow \mu_t - \gamma \nabla_\mu [\mathcal{L}_{CE} + \underline{\lambda_{\mathcal{F}}} + \theta(c(\mu, \Sigma_t)) + \frac{\rho}{2}\|c(\mu, \Sigma_t)\|_F^2]$
$\sigma_{t+1} \leftarrow \sigma_t - \gamma \nabla_\sigma [\mathcal{L}_{CE} + \underline{\lambda_{\mathcal{F}}} + \theta(c(\mu_t, \Sigma)) + \frac{\rho}{2}\|c(\mu_t, \Sigma)\|_F^2]$
Optimize θ with $\theta_{t+1} \leftarrow \theta_t + \rho \cdot c(\mu_t, \Sigma_t)$
end while
Sample the normal and reduction cell based on sampled α and β
Retrain the searched architecture from scratch on training set

a min-max problem so that (16) can be rewritten as

$$\min_{\mu^\alpha, \Sigma^\alpha, \mu^\beta, \Sigma^\beta} \max_\theta \mathcal{L}_{CE} + \underline{\lambda_{\mathcal{F}}} + \theta(c(\mu, \Sigma)) + \frac{\rho}{2}\|c(\mu, \Sigma)\|_F^2,$$
$$c(\mu, \Sigma) = \mu + \Phi^{-1}(\eta)\sigma^2 - ln(\overline{\lambda_{\mathcal{F}}^*}), \tag{17}$$

where θ is the dual variable and ρ is positive number predefined in ADMM. The first step is to update μ while fixing other variables and the second step is to update σ while fixing other variables as

$$\mu_{t+1} \leftarrow \mu_t - \gamma \nabla_\mu [\mathcal{L}_{CE} + \underline{\lambda_{\mathcal{F}}} + \theta(c(\mu, \Sigma_t)) + \frac{\rho}{2}\|c(\mu, \Sigma_t)\|_F^2],$$
$$\sigma_{t+1} \leftarrow \sigma_t - \gamma \nabla_\sigma [\mathcal{L}_{CE} + \underline{\lambda_{\mathcal{F}}} + \theta(c(\mu_t, \Sigma)) + \frac{\rho}{2}\|c(\mu_t, \Sigma)\|_F^2], \tag{18}$$

where $\mu^\alpha, \Sigma^\alpha, \mu^\beta, \Sigma^\beta$ are updated through back-propagation. The dual variable θ is updated with learning rate of ρ as

$$\theta_{t+1} \leftarrow \theta_t + \rho \cdot c(\mu_t, \Sigma_t) \tag{19}$$

The entire Robust Architecture search with Confidence Learning algorithm denoted as RACL, is shown in Algorithm 1. With the proposed algorithm, we impose confidence learning on the values of architecture parameters α and β, which strengthens the confidence of robust architecture sampling. Compared with previous NAS without confidence learning, RACL relaxes the procedure of sampling by treating architecture parameters as distributions, instead of directly using *argmax* which leads to a gap between searching and sampling. By incorporating the confidence learning of robust architecture sampling into the NAS algorithm, RACL could discover the broad landscape of robust neural architectures.

IV. Experiments

In this section, we conduct a series of experiments to empirically demonstrate the effectiveness of the proposed RACL algorithm. We mainly consider two scenarios, including the performance with adversarial training (AT) and the one without AT. To evaluate the effectiveness of RACL with AT, we adversarially train the searched neural architecture and compare it with various neural architectures searched by NAS algorithms as well as state-of-the-art network architectures. To evaluate the effectiveness of RACL without AT, we retrain the searched neural architecture and compare it with other NAS algorithms. Furthermore, we consider the evaluation on NAS-Bench-201 [55]. We show that under various adversarial attack settings, the robust neural architectures searched by RACL always achieve better robustness than other baselines.

A. Experimental Setup

Neural Architecture Search Setup: Following previous works [17], [19], we search the robust neural architectures on CIFAR-10 dataset which contains 50 K training images and 10 K validation images over 10 classes. During the searching phase, the training set is divided into two parts with equal sizes for architecture and weight optimization respectively. The search space includes 8 candidates: 3×3 and 5×5 separable convolutions, 3×3 and 5×5 dilated separable convolutions, 3×3 max pooling, 3×3 average pooling, skip connection, and zero, as suggested by previous works [17], [19]. The supernet is constructed by stacking 8 cells including 6 normal cells and 2 reduction cells, each of which contains 6 nodes. For the training settings, we follow the setups of PC-DARTS [17]. The searching phase takes 50 epochs with a batch size of 128. We use SGD with momentum. The initial learning rate is 0.1 with a momentum of 0.9 and a weight decay is 3×10^{-4} to update the supernet weights. Architecture parameters were updated with Adam with a learning rate of 6×10^{-4} and a weight decay of 1×10^{-3}. The searching time of RACL takes 0.5 GPU days.

Benchmarks and Retrain Details With AT: We extensively evaluate the proposed algorithm on four datasets including CIFAR-10, CIFAR-100, Tiny-ImageNet, and ImageNet, which are widely compared by other previous work. The searched superior neural architecture is sampled based on the proposed sampling strategy. Following the setting in NAS [17], [19], we stack searched cells to form a 20-layer network and retrained it with the entire training set. For the evaluation stage, we adopt the popular adversarial training framework FAT [70] to retrain all the baselines. We train the network from scratch for 100 epochs with a batch size of 128 on the entire training set. We use SGD optimizer with an initial learning rate of 0.1, momentum of 0.9, and a weight decay of 2×10^{-4}. The norm gradient clipping is set to 5. Following [71], the hyperparameter which balances the adversarial loss and KL divergence is set to 6. Since we focus on the impact of architecture on adversarial robustness, we train the searched architectures as well as state-of-the-art network architectures with the same adversarial training setting. Through training these architectures in the same adversarial manner, we conduct a fair comparison

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

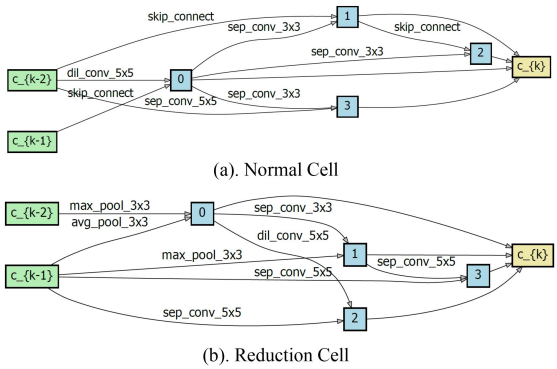


Fig. 3. The visualization of normal and reduction cells searched by RACL are shown in (a) and (b).

among different architectures and demonstrate how they improve or constrain the adversarial robustness. For CIFAR-10 and CIFAR-100, we use adversarial training with the total perturbation size $\epsilon = 8/255$. The maximum number of attack iterations is set to 10 with a step size of $2/255$. For Tiny-ImageNet, we set $\epsilon = 4/255$. The maximum number of attack iterations is set to 6 with a step size of $2/255$. For ImageNet, we set $\epsilon = 4/255$ with a maximum attack iterations of 2 and train the network for 50 epochs. An illustration of the searched normal and reduction cells is shown in Fig. 3. More analysis on searched robust neural architectures will be covered in Section IV-F.

Benchmarks and Retrain Details Without AT: To verify the contribution to adversarial robustness from architecture alone, we evaluate the proposed algorithm on CIFAR-10 and CIFAR-100 without adversarial training. Following the retrain setting in PC-DARTS [17], we train the network from scratch for 600 epochs with a SGD optimizer. The initial learning rate is set to 0.025, momentum to 0.9, and weight decay to 3×10^{-4}. To further evaluate the effectiveness of RACL, we consider a popular NAS benchmark NAS-Bench-201 [55] which contains 15,625 neural cell candidates within a pre-defined search space. It includes all possible architectures generated by 4 nodes and 5 associated operation options. For a fair comparison of adversarial robustness, we perform searching via different NAS algorithms and then directly load their corresponding network weights from NAS-Bench-201.

B. Evaluation With Adversarial Training

Existing approaches mainly focus on adversarial training, however, the impact of architecture is always ignored. In fact, the architecture design could significantly influence the adversarial robustness. Since adversarial training is the most popular defense technique and is also orthogonal to architecture design, we evaluate the searched architecture from different NAS algorithms after adversarial training to highlight the necessity of robust architecture search, which follows the evaluation setting in RobNet [13]. In this subsection, we demonstrate that constructing networks via the neural architectures searched by RACL can further improve robust accuracy after adversarial training. For robustness evaluation, we choose various popular powerful attacks including Fast Gradient Sign Method (FGSM) [22], Momentum Iterative Method (MIM) [31], Projected Gradient Descent (PGD) [32], CW attack [72], and Auto Attack [73]. Consistent with previous adversarial literature [32], [46], the perturbation is considered under l_∞ norm with the total perturbation size of $8/255$ on CIFAR-10/100 and $4/255$ on Tiny-ImageNet and ImageNet. For CW attack, the steps are set to 1000 with a learning rate of 0.01.

Comparison With Human-Designed Architectures and NAS Algorithms on CIFAR-10: For an extensive evaluation of adversarial robustness with different architecture designs, we take into consideration three types of algorithms, including popular human-designed architectures, standard NAS algorithms, and robust NAS algorithms. Specifically, we include ResNet-18 [3] and DenseNet-121 [4], which are widely compared in adversarial robustness benchmarks. Besides, we include popular standard NAS algorithms which share the same search space, including AmoebaNet [66], NASNet [53], DARTS [17], and PC-DARTS [19]. Furthermore, some robust NAS algorithms targeting the adversarial robustness are also included for comparison, including RobNet [13], ABanditNAS [56], AdvRush [58], and DSRNA [59]. The results are shown in Table I. Compared with these human-designed architectures, RACL shows obvious superiority of robust accuracy over all the baselines under various attacks with fewer parameters. Compared with standard NAS algorithms, RACL achieves better adversarial accuracy than other state-of-the-art neural architectures on CIFAR-10. For example, compared with baseline PC-DARTS, though both RACL and PC-DARTS achieve similar clean accuracy and model size, their performance with adversarial training varies differently. RACL achieves an accuracy of 62.55% under FGSM attack, with 1.47% improvement ($61.08\% \rightarrow 62.55\%$) over that of PC-DARTS, and 2.96% improvement ($52.36\% \rightarrow 55.32\%$) over that of PC-DARTS under PGD^{100} attack. These standard NAS algorithms cannot achieve satisfactory robust accuracy under strong attacks, such as Auto Attack. On the contrary, these robust NAS algorithms, such as RobNet-free and AdvRush, achieve higher accuracy under strong Auto Attack. Compared with robust NAS algorithms, RACL still achieves the best adversarial robustness in most scenarios. For example, RACL achieves an accuracy of 60.00% under MIM attack, with 7.05% improvement ($52.95\% \rightarrow 60.00\%$) over that of RobNet-free. Overall, RACL achieves superior trade-offs among parameters, clean accuracy, and adversarial accuracy, which highlights the effectiveness of RACL.

Evaluation on CIFAR-100, Tiny-ImageNet, and ImageNet: Besides CIFAR-10, we further evaluate the adversarial robustness of different NAS algorithms on other datasets, including CIFAR-100, Tiny-ImageNet, and ImageNet. Regarding the baselines, we include AmoebaNet [66], NASNet [53], DARTS [17], and PC-DARTS [19]. For a fair comparison, we retrain the searched cells using FAT adversarial training for all the models to verify the robustness of RACL. The detailed evaluation results are shown in Tables II and III. The best result for each column is highlighted in bold. As shown in Table II, RACL achieves the best robust accuracy than other baselines under different attacks on CIFAR-100

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EVALUATION OF RACL ADVERSARIAL ROBUSTNESS ON CIFAR-10 COMPARED WITH OTHER HUMAN-DESIGNED ARCHITECTURES AND OTHER ROBUST NAS ALGORITHMS UNDER WHITE-BOX ATTACKS

Category	Model	Params	Natural	FGSM	PGD^{20}	PGD^{100}	MIM	AA
Hand-Crafted	ResNet-18 [3]	11.2M	84.09%	54.64%	45.86%	45.53%	45.23%	43.22%
	DenseNet-121 [4]	7.0M	85.95%	58.46%	50.49%	49.92%	48.19%	47.46%
Standard NAS	AmoebaNet [66]	3.2M	82.28%	59.12%	53.69%	53.34%	57.26%	47.88%
	NASNet [53]	3.8M	84.37%	61.38%	53.35%	52.84%	58.72%	48.19%
	DARTS [17]	3.3M	80.65%	59.63%	54.04%	53.73%	57.55%	48.13%
	PC-DARTS [19]	3.6M	84.32%	61.08%	53.01%	52.36%	58.10%	47.95%
Robust NAS	ABanditNAS-10 [56]	5.2M	**90.64%**	**81.31%**	50.51%	45.73%	54.19%	16.03%
	RobNet-large [13]	6.9M	78.57%	54.98%	49.44%	49.24%	49.92%	45.82%
	RobNet-free [13]	5.6M	85.00%	59.22%	52.09%	51.14%	52.95%	48.56%
	AdvRush [58]	4.2M	85.59%	59.98%	52.76%	52.55%	-	49.28%
	DSRNA [59]	2.0M	80.93%	54.49%	49.11%	48.89%	-	44.87%
	RACL(ours)	3.3M	84.04%	62.55%	**55.68%**	**55.32%**	**60.00%**	**50.07%**

PGD^{20} denotes PGD attack with 20 iterations. The best results are in bold and the second best is underlined.

TABLE II
EVALUATION OF RACL ADVERSARIAL ROBUSTNESS ON CIFAR-100 AND TINY-IMAGENET COMPARED WITH VARIOUS NAS ALGORITHMS UNDER WHITE-BOX ATTACKS

Dataset	Model	Params	Natural	FGSM	MIM	PGD^{20}	PGD^{100}	CW	AutoAttack
CIFAR-100	AmoebaNet	3.2M	56.51%	32.67%	31.44%	29.70%	29.66%	49.03%	25.26%
	NASNet	3.8M	57.97%	31.54%	30.14%	28.58%	28.44%	49.64%	24.42%
	DARTS	3.3M	**58.67%**	32.71%	31.14%	29.21%	29.11%	50.32%	24.30%
	PC-DARTS	3.6M	57.20%	31.85%	30.46%	28.62%	28.50%	49.40%	24.10%
	RACL(ours)	3.3M	57.83%	**33.89%**	**32.41%**	**30.41%**	**30.15%**	**52.56%**	**25.55%**
Tiny-ImageNet	AmoebaNet	3.2M	47.84%	31.44%	30.57%	30.12%	30.09%	42.56%	25.20%
	NASNet	3.8M	47.85%	30.76%	29.80%	29.47%	29.44%	41.93%	26.17%
	DARTS	3.3M	48.20%	31.38%	30.71%	30.30%	30.25%	42.23%	25.09%
	PC-DARTS	3.6M	47.24%	30.04%	29.18%	28.55%	28.53%	40.91%	23.15%
	RACL(ours)	3.3M	**48.86%**	**31.98%**	**31.12%**	**30.63%**	**30.63%**	**42.99%**	**27.91%**

Best results in bold.

TABLE III
ADVERSARIAL ROBUSTNESS EVALUATION OF NAS ALGORITHMS WITH ADVERSARIAL TRAINING ON IMAGENET

Model	Params	Clean	FGSM	PGD^{20}	PGD^{100}
AmoebaNet	4.6M	47.61%	31.15%	29.78%	29.74%
NASNet	5.6M	49.66%	32.55%	31.09%	31.06%
DARTS	4.7M	48.16%	31.52%	30.01%	29.97%
PC-DARTS	5.3M	49.45%	32.52%	31.01%	30.97%
RACL(ours)	4.8M	**49.74%**	**32.84%**	**31.24%**	**31.24%**

TABLE IV
ADVERSARIAL ROBUSTNESS EVALUATION OF DIFFERENT SEARCHED ARCHITECTURES WITH STANDARD TRAINING

Dataset	Model	Natural	PGD^{20}	PGD^{100}	MIM	AA
C-10	AmoebaNet	96.99%	43.71%	42.67%	45.00%	31.05%
	NASNet	96.94%	46.66%	45.95%	46.28%	31.19%
	DARTS	**97.03%**	37.11%	36.75%	36.96%	24.61%
	PC-DARTS	96.97%	45.06%	44.38%	46.84%	32.87%
	RACL(ours)	95.71%	**49.84%**	**49.35%**	**54.69%**	**40.48%**
C-100	AmoebaNet	**81.09%**	21.08%	20.61%	18.74%	13.20%
	NASNet	80.41%	21.28%	20.82%	18.70%	13.08%
	DARTS	81.03%	19.00%	18.17%	16.02%	11.03%
	PC-DARTS	80.79%	21.09%	20.38%	19.53%	13.09%
	RACL(ours)	77.77%	**21.40%**	**20.89%**	**22.45%**	**13.76%**

and Tiny-ImageNet. For example, RACL achieves an accuracy of 25.55% under Auto Attack, with 1.25% improvement ($24.30\% \rightarrow 25.55\%$) over that of DARTS on CIFAR-100. On Tiny-Imagenet, RACL achieves an accuracy of 42.99% under CW attack, with 1.06% improvement ($41.93\% \rightarrow 42.99\%$) over that of NASNet. Similarly on ImageNet, as shown in Table III, RACL achieves an accuracy of 31.24% under PGD^{100}, with 1.27% improvement ($29.97\% \rightarrow 31.24\%$) with over that of DARTS. We empirically show that RACL consistently achieves the best robust performance compared with other NAS algorithms with the same search space under various attacks, which indicates that the adversarial robustness can be further improved by RACL.

C. Evaluation Without Adversarial Training

In the previous subsection, we explore the effectiveness of RACL after adversarial training. To further verify the effectiveness of robust architecture search, we remove the adversarial training and directly retrain the searched architecture with clean images to evaluate how different neural architectures influence the adversarial robustness with standard training. Specifically, we consider two benchmarks. First, we follow the standard retrain setting of PC-DARTS [17] to train all the baselines on CIFAR-10 and CIFAR-100. Second, we perform searching via different NAS algorithms within the search space of NAS-bench-201 [55] and directly load their corresponding network weights to evaluate their adversarial robustness. For the evaluation, we set the total perturbation size to be $1/255$ for PGD, MIM, and AA attacks.

Comparison of Standard Trained Neural Architectures: For comparison, we include popular NAS algorithms, including AmoebaNet, NASNet, DARTS, and PC-DARTS. The evaluation includes robust accuracy under PGD, MIM, and Auto Attack. The results are shown in Table IV. Without adversarial training, RACL shows superiority over other baselines on CIFAR-10. For example, RACL achieves an accuracy of 40.48% under Auto Attack, with 7.61% improvement ($32.87\% \rightarrow 40.48\%$) over that of PC-DARTS on CIFAR-10. Similarly, RACL achieves the highest robust accuracy under all the attacks on CIFAR-100.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ADVERSARIAL ROBUSTNESS EVALUATION OF DIFFERENT NAS ALGORITHMS WITH STANDARD TRAINING ON NAS-BENCH-201

Dataset	Search Alg	Natural	PGD^{20}	PGD^{100}	MIM	AA
C-10	ENAS	34.80%	29.33%	29.33%	29.32%	27.85%
	GDAS	**85.45%**	28.75%	28.57%	28.57%	21.75%
	DARTS	36.84%	29.43%	29.43%	29.42%	27.16%
	Reinforce	85.24%	21.64%	21.07%	19.60%	15.51%
	Random	77.51%	24.51%	24.18%	23.13%	20.60%
	RACL(ours)	68.53%	**30.98%**	**30.93%**	**29.81%**	**27.88%**
C-100	ENAS	9.87%	8.26%	8.26%	8.27%	7.27%
	GDAS	**52.13%**	11.94%	11.85%	12.18%	9.77%
	DARTS	10.05%	8.71%	8.70%	8.70%	7.49%
	Reinforce	50.21%	13.31%	13.08%	12.55%	10.05%
	Random	41.44%	9.26%	9.08%	8.80%	6.63%
	RACL(ours)	28.75%	**14.11%**	**14.07%**	**13.67%**	**11.27%**

For example, RACL achieves an accuracy of 22.45% under MIM attack, with 3.71% improvement ($18.74\% \rightarrow 22.45\%$) over that of AmoebaNet on CIFAR-100. Although the robust accuracy in Table IV may not be competitive with other adversarially trained models, the comparison with other NAS algorithms without AT provides strong evidence that RACL can discover naturally robust neural architectures.

Evaluation on NAS-Bench-201: We further verify the searching effectiveness of RACL on a popular benchmark NAS-bench-201 [55], where all the possible architecture candidates within a predefined search space and their corresponding retrained weights are available for a fair comparison. To demonstrate the superiority of RACL, we include several NAS algorithms for comparison, including ENAS [52], GDAS [18], DARTS [17], reinforcement [14], and random search that we select the best run out of 4 samplings. After searching the optimal architectures via different algorithms on CIFAR-10 and 100, we directly load their weights from NAS-bench-201 for adversarial robustness evaluation. We consider several attack settings, including PGD^{20}, PGD^{100}, MIM, and Auto Attack. The results are shown in Table V. It is obvious that RACL can always discover the neural architectures that achieves superior adversarial robustness. Specifically, compared with GDAS, although the architecture searched by RACL has lower natural accuracy, the robust accuracy under various attacks is much higher. For example, RACL achieves an accuracy of 27.88% under Auto attack, with 6.13% improvement ($21.75\% \rightarrow 27.88\%$) over that of GDAS on CIFAR-10, and achieves an accuracy of 14.07% under PGD^{100} attack, with 2.22% improvement ($11.85\% \rightarrow 14.07\%$) over that of GDAS on CIFAR-100. Thus, the evaluation on NAS-Bench-201 also provides strong evidence that RACL can discover adversarially robust architecture within a pre-defined search space.

D. Against Closed-Box Attacks

Transfer-Based Closed-Box Attack Evaluation: We next evaluate the robustness of RACL under closed-box attacks. Following previous literature [32], [74], we apply transfer-based closed-box attacks which generate adversarial samples using a victim model and feed them to the target models. In this paper, we take a ResNet-110 network as the victim model. The transferred adversarial samples are generated through FGSM, MIM, and PGD attacks. The adversarial accuracy of different architectures is compared after they are fed with these transferred adversarial samples, as shown in Table VI. Compared with other standard NAS algorithms, RACL achieves the highest robust accuracy in all the scenarios under these transfer-based attacks, which highlights the adversarial robustness of the proposed algorithm against transfer-based closed-box attacks.

TABLE VI
EVALUATION OF RACL ADVERSARIAL ROBUSTNESS ON CIFAR-10 UNDER TRANSFER-BASED CLOSED-BOX ATTACK SETTING

Model	FGSM	MIM	PGD^{20}
AmoebaNet	81.92%	82.04%	82.61%
NasNet	82.32%	82.60%	83.21%
DARTS	78.76%	78.83%	79.29%
PC-DARTS	82.47%	82.65%	83.06%
RACL(ours)	**82.78%**	**82.92%**	**83.45%**

TABLE VII
TRANSFERABILITY TEST ON CIFAR-10 AMONG DIFFERNET MODELS USING PGD ATTACK

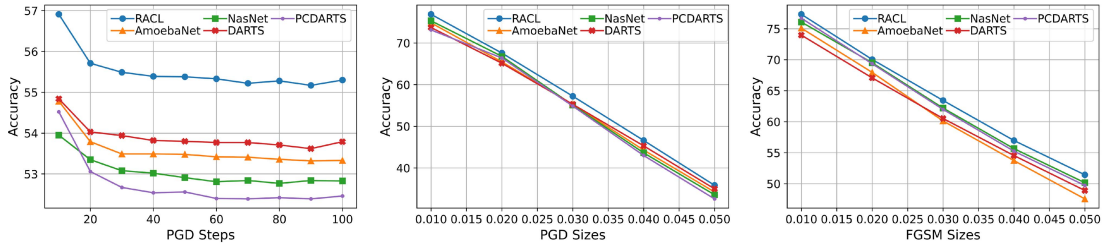
Source \ Target	AmoebaNet	NasNet	DARTS	PC-DARTS	ours
AmoebaNet	53.69	66.79	64.39	66.50	**67.21**
NasNet	64.77	53.35	64.31	65.62	**66.22**
DARTS	65.03	66.91	54.04	66.74	**67.04**
PC-DARTS	64.37	65.45	63.91	53.01	**66.23**
RACL(ours)	64.49	**66.24**	64.06	65.90	55.68

The best results in each row are in bold. Underline denotes the white-box robustness.

Transferability Test on CIFAR-10 Under PGD Attack: Following [13], we further conduct the transferability test on CIFAR-10. We use different NAS algorithms as source models to generate adversarial samples through 10-iteration PGD attack and feed them to other target models as cross closed-box attacks. The results are shown in Table VII. Each row denotes the robust accuracy of different target models under the closed-box attack from the same source model. Correspondingly, each column denotes the robustness of a target model under attack from different source models. Comparing each row, RACL achieves the best accuracy under the attacks from different source models, which indicates that although these architectures are searched within the same search space, they show different robustness under attacks. The large gap between RACL and other baselines also highlights the superiority of RACL under closed-box settings. Furthermore, by comparing the transferability between each model pair, RACL tends to generate stronger adversarial samples. E.g., RACL $\rightarrow$ AmoebaNet achieves the successful attack success rate of 35.52% and AmoebaNet $\rightarrow$ ours achieves the successful attack success rate of 29.59%. Taking NASNet, DARTS, and PC-DARTS as target models, RACL generates the adversarial samples that achieve the highest attack success rate except for the white-box attack.

Transferability Test on CIFAR-10 Under RFGSM Attack: Furthermore, we provide additional robustness evaluation of RACL through a transferability test on CIFAR-10 under RFGSM attack [75]. The detailed results are shown in Table VIII. The underline denotes the adversarial accuracy under white-box RFGSM attack where the total perturbation is set to $8/255$. Comparing the accuracy on the diagonal, RACL achieves the best white-box performance under RFGSM attack. Each row

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.



(a). Adversarial accuracy on CIFAR-10 under different PGD iterations (b). Adversarial accuracy on CIFAR-10 under different PGD total perturbation size (c). Adversarial accuracy on CIFAR-10 under different FGSM total perturbation size

Fig. 4. Robustness evaluation under different perturbation sizes and attack iterations.

TABLE VIII
TRANSFERABILITY TEST ON CIFAR-10 AMONG DIFFERENT MODELS UNDER RFGSM ATTACK

Source \ Target	AmoebaNet	NasNet	DARTS	PC-DARTS	ours
AmoebaNet	76.68	80.97	77.13	80.93	**81.22**
NasNet	78.93	78.52	77.18	80.85	**81.09**
DARTS	78.81	80.92	75.26	80.82	**81.16**
PC-DARTS	78.70	80.68	77.02	78.43	**81.16**
RACL(ours)	78.78	80.73	77.18	**80.83**	78.83

The best results in each row are in bold. Underline denotes the white-box robustness.

denotes the robustness of different target models under the closed-box attack from the same source model. Comparing each column, RACL shows strong adversarial transferability as the source model. Comparing each row, RACL achieves better closed-box adversarial accuracy in all the scenarios as shown in Table VIII with bold. *E.g.*, as shown in the fourth row, PC-DARTS $\rightarrow$ RACL achieves the successful attack success rate of 18.84%, PC-DARTS $\rightarrow$ AmoebaNet of 21.30%, PC-DARTS $\rightarrow$ NASNet of 19.32% and PC-DARTS $\rightarrow$ DARTS of 22.98%. Similarly, RACL achieves strong adversarial transferability with RFGSM attack. Thus, RACL shows superior adversarial robustness against different transferred-based attacks, demonstrating our algorithm's effectiveness.

E. Robustness Under Various Perturbation Size and Attack Iterations

Robustness Under Increasing Attack Iterations: We further conduct experiments with different white-box attack parameters, including the size of the perturbation and the number of iterations. Following [13], we strengthen the adversarial attack by boosting the attack iterations to 100 for the PGD attack with a step size of $2/255$. The comparison with other baselines is shown in Fig. 4(a) where RACL consistently achieves the best accuracy for different PGD iterations. Furthermore, RACL shows a relatively stronger defense capability against PGD attacks with more iterations. For example, NasNet achieves 53.35% under PGD^{20} and 52.83% under PGD^{100} with a gap of 0.52%. In contrast, RACL achieves 55.68% under PGD^{20} and 55.32% under PGD^{100} with a gap of 0.36%, which demonstrates that RACL can better remain the robustness after more attack iterations. Compared to RobNet family with the same search space on PGD^{100} [13], RACL achieved better performance with fewer parameters than RobNet-small, RobNet-medium, RobNet-large, and RobNet-free but 7.25%, 6.36%, 6.08%, and 2.75% accuracy improvement respectively, which also shows the efficiency of RACL.

Robustness Under Increasing Perturbation Size: Besides attack iterations, we evaluate the adversarial robustness under different perturbation budgets. As shown in Fig. 4(b) and (c), the total perturbation size ranges from 0.01 to 0.05 for both PGD and FGSM attacks. Our proposed RACL algorithm always performs better than other baselines under different perturbation budgets, illustrating that RACL can provide a stronger defense against various adversarial attacks. Similarly, our advantage becomes more obvious when the attack is allowed with a larger total perturbation size. *E.g.*, comparing NasNet with RACL on the $\text{PGD}_{0.01}$ and $\text{PGD}_{0.05}$, the gap increases 0.71% when the attack size grew ($75.33\% \rightarrow 76.89\%$ on $\text{PGD}_{0.01}$ and $33.57\% \rightarrow 35.84\%$ on $\text{PGD}_{0.05}$); comparing AmoebaNet with RACL on the $\text{FGSM}_{0.01}$ and $\text{FGSM}_{0.05}$, the gap increases 1.65% ($75.10\% \rightarrow 77.33\%$ on $\text{FGSM}_{0.01}$ and $47.55\% \rightarrow 51.43\%$ on $\text{FGSM}_{0.05}$), which highlights the adversarial robustness of RACL within a wider perturbation space for various attacks.

F. Potential Pattern and Variance of RACL

Visualization of Searched Cells: Besides the cells visualized in Fig. 3, we run RACL several times to explore the potential patterns RACL tended to discover and provided more insights into the searched robust neural architectures. More searched architectures are given in Fig. 5. Together with the searched cells in Fig. 3, we showed that there exist some potential patterns that RACL prefers. There always exists a ResNet-like pattern in the searched normal cells. For example, the input of each node tends to be a combination of skip connection and another operation with trainable parameters, such as the Node 0, 1, 2 in Figs. 3(a) and 5(c). Besides the ResNet-like pattern in the normal cells, RACL tends to select pooling layers such as 3×3 max pooling instead of skip connection in the reduction cells, as shown in Fig. 3(b) and Fig. 5(b) and (d). Overall, the searched cells of RACL look like a tuned version of ResNet. The potential pattern is also reasonable from the perspective of RACL algorithm. Since RACL aims to search the architecture with a smaller Lipschitz constant with high confidence, it is more likely for RACL to sample the operations with lower Lipschitz constants, such as pooling layers and skip connections. However, directly

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

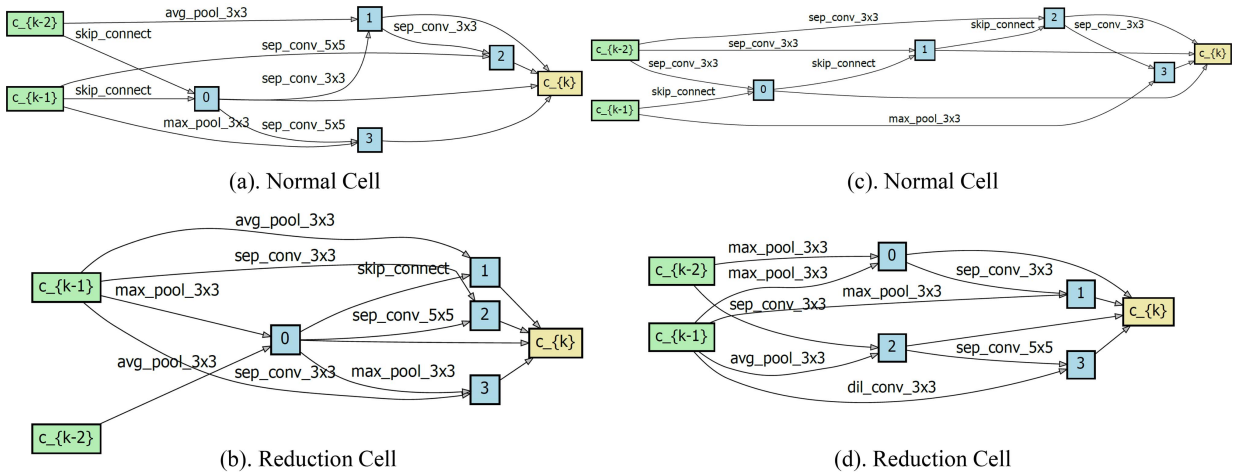


Fig. 5. The visualization of cells searched by RACL through multiple runs.

TABLE IX
MULTIPLE RUNS OF SEARCHED CELLS WITH ADVERSARIAL TRAINING

Models	Clean	FGSM	PGD^{20}	MIM	CW	AA
AmoebaNet	81.86±0.22	59.19±0.32	53.43±0.45	57.07±0.49	78.33±0.63	47.64±0.37
NasNet	**84.05±0.64**	59.49±0.53	53.37±0.70	58.16±0.55	79.57±0.74	48.34±0.46
DARTS	80.84±0.88	59.49±0.53	53.82±0.59	57.32±0.45	77.34±0.99	48.31±0.38
PC-DARTS	84.01±0.68	60.85±0.26	53.30±0.51	58.17±0.46	79.93±0.56	47.56±0.50
RACL	83.98±0.22	**62.48±0.10**	**55.50±0.38**	**60.01±0.39**	**80.11±0.39**	**50.14±0.33**

The mean of clean or adversarial accuracy is reported with its error bar.

TABLE X
ABLATION ANALYSIS OF RACL WITH RESPECT TO CONFIDENCE LEARNING, ρ, AND η

Setting	Clean	FGSM	PGD^{20}	MIM	CW	AA
Random Search	81.55	58.73	51.36	55.72	76.63	46.82
w/o Gradient Norm	82.54	60.33	53.87	58.04	78.94	48.25
w/o CL	**84.35**	61.64	54.58	58.68	78.72	48.89
$\eta = 0.9, \rho = 0.01$	83.06	61.60	55.66	59.34	79.51	48.98
$\eta = 0.7, \rho = 0.001$	84.30	61.58	55.00	59.14	80.36	49.03
$\eta = 0.9, \rho = 0.001$	84.04	**62.07**	**55.68**	**60.00**	**80.90**	**50.07**

selecting operations without trainable parameters could significantly reduce the natural accuracy, which also reduce robust accuracy. Thus, there exists a trade-off between operations that benefit robustness and those that benefit classification performance. Through the constrained optimization via RACL, it is natural to discover a tuned version of ResNet with better robust accuracy.

Robustness Stability of Searched Cells: To further demonstrate the effectiveness of RACL, we report the error bar of RACL as well as other baselines through multiple runs to evaluate the robustness stability of searched cells. Following previous NAS work [17], we retrain all the baselines multiple times and report the performance of neural architectures searched by different NAS algorithms and RACL (out of 5 runs). The detailed results are shown in Table IX. For each algorithm, we report the average clean and robust accuracy with the standard error. Comparing each column, RACL consistently achieves the best average robust accuracy under various attacks after multiple runs. For example, RACL achieves an average accuracy of 50.14% under Auto Attack and 55.50% under PGD^{20}, which is around 2% higher than other baselines. For the error bar, RACL has smaller fluctuation in almost all the scenarios among different NAS algorithms, which demonstrates that RACL can discover robust neural architectures with better robustness and stability.

G. Ablation Analysis

In this section, we conduct ablation studies on the hyperparameters of RACL algorithm as well as confidence learning. The ablation study results are shown in Table X. We first apply the random search algorithm within the pre-defined search space to rule out the possibility that the major improvement comes from the search space. We randomly sampled 10 models and selected the best one for comparison. We then remove the confidence learning and apply the constraint in (10) to evaluate the effectiveness of confidence learning. Similarly, we remove the gradient norm constraint in (16) to evaluate the effectiveness of lower bound constraint. Comparing the first and other rows, the random search algorithm cannot achieve competitive results within a pre-defined search space, which demonstrates the necessity of discovering robust neural architectures. Comparing the second and last row, the searched architecture without confidence learning tends to have relatively higher natural accuracy. On the contrary, our proposed RACL achieves a relatively large increment in adversarial accuracy with confidence learning, which highlights the importance of proposed confident architecture sampling. We then investigated the influence of hyperparameter ρ and reported the performance of searched robust cell on CIFAR-10 under different values of ρ. Through comparison, ρ with a large value could hurt the classification performance on clean images. On the contrary, ρ with a small value reduces the influence of Lipschitz constraint and results in inferior adversarial accuracy. The influence of confidence hyperparameter η is also investigated. From (16), η controls the balance between the mean and variance of Lipschitz constant $\overline{\lambda_{\mathcal{F}}}$. Through cross-validation, η is set to 0.9 to obtain the best adversarial accuracy.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

In this paper, we propose to tackle the vulnerability of neural networks by incorporating NAS frameworks. Through sampling architecture parameters from trainable log-normal distributions, we show that the approximated Lipschitz constant of the entire network can be formulated as a univariate log-normal distribution, which enables the proposed algorithm, Robust Architecture with Confidence Learning to form confidence learning of architecture parameters on the robustness through a Lipschitz constraint. Thorough experiments demonstrate the influence of architecture on adversarial robustness and the effectiveness of RACL under various attacks on different datasets.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105. [Online]. Available: http://dl.acm.org/citation.cfm?id=2999134.2999257

[2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.155*. [Online]. Available: http://arxiv.org/abs/1409.1556

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," 2016, *arXiv:1603.05027*. [Online]. Available: http://arxiv.org/abs/1603.05027

[4] G. Huang, Z. Liu, and K. Q. Weinberger, "Densely connected convolutional networks," 2016, *arXiv:1608.06993*. [Online]. Available: http://arxiv.org/abs/1608.06993

[5] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.

[6] H. Chen et al., "Distilling portable generative adversarial networks for image translation," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 3585–3592.

[7] A. M. Nguyen, J. Yosinski, and J. Clune, "Deep neural networks are easily fooled: High confidence predictions for unrecognizable images," 2014, *arXiv:1412.1897*. [Online]. Available: http://arxiv.org/abs/1412.1897

[8] S. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: A simple and accurate method to fool deep neural networks," 2015, *arXiv:1511.04599*. [Online]. Available: http://arxiv.org/abs/1511.04599

[9] S. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, and P. Frossard, "Universal adversarial perturbations," 2016, *arXiv:1610.08401*. [Online]. Available: http://arxiv.org/abs/1610.08401

[10] B. Biggio et al., "Evasion attacks against machine learning at test time," 2017, *arXiv:1708.06131*. [Online]. Available: http://arxiv.org/abs/1708.06131

[11] C. Xie, Y. Wu, L. van der Maaten, A. L. Yuille, and K. He, "Feature denoising for improving adversarial robustness," 2018, *arXiv:1812.03411*. [Online]. Available: http://arxiv.org/abs/1812.03411

[12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105. [Online]. Available: http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf

[13] M. Guo, Y. Yang, R. Xu, Z. Liu, and D. Lin, "When NAS meets robustness: In search of robust architectures against adversarial attacks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 631–640.

[14] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," 2016, *arXiv:1611.0157*. [Online]. Available: http://arxiv.org/abs/1611.01578

[15] B. Baker, O. Gupta, N. Naik, and R. Raskar, "Designing neural network architectures using reinforcement learning," 2016, *arXiv:1611.02167*. [Online]. Available: http://arxiv.org/abs/1611.02167

[16] I. Bello, B. Zoph, V. Vasudevan, and Q. V. Le, "Neural optimizer search with reinforcement learning," 2017, *arXiv:1709.07417*. [Online]. Available: http://arxiv.org/abs/1709.07417

[17] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," 2018, *arXiv:1806.09055*. [Online]. Available: http://arxiv.org/abs/1806.09055

[18] X. Dong and Y. Yang, "Searching for a robust neural architecture in four GPU hours," 2019, *arXiv:1910.04465*.

[19] Y. Xu et al., "PC-DARTS: Partial channel connections for memory-efficient differentiable architecture search," 2019, *arXiv:1907.05737*. [Online]. Available: http://arxiv.org/abs/1907.05737

[20] Y. Tang et al., "A semi-supervised assessor of neural architectures," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1810–1819.

[21] A. Zela, T. Elsken, T. Saikia, Y. Marrakchi, T. Brox, and F. Hutter, "Understanding and robustifying differentiable architecture search," in *Proc. Int. Conf. Learn. Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=H1gDNyrKDS

[22] C. Szegedy et al., "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.

[23] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.

[24] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial machine learning at scale," 2016, *arXiv:1611.01236*. [Online]. Available: http://arxiv.org/abs/1611.01236

[25] C. Xiao, B. Li, J. Zhu, W. He, M. Liu, and D. Song, "Generating adversarial examples with adversarial networks," 2018, *arXiv:1801.02610*. [Online]. Available: http://arxiv.org/abs/1801.02610

[26] N. Papernot, P. D. McDaniel, I. J. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against deep learning systems using adversarial examples," 2016, *arXiv:1602.02697*. [Online]. Available: http://arxiv.org/abs/1602.02697

[27] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, "Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models," in *Proc. 10th ACM Workshop Artif. Intell. Secur.*, 2017, pp. 15–26. [Online]. Available: http://dx.doi.org/10.1145/3128572.3140448

[28] A. Ilyas, L. Engstrom, A. Athalye, and J. Lin, "Query-efficient black-box adversarial examples," 2017, *arXiv:1712.07113*. [Online]. Available: http://arxiv.org/abs/1712.07113

[29] S. Moon, G. An, and H. O. Song, "Parsimonious black-box adversarial attacks via efficient combinatorial optimization," 2019, *arXiv:1905.06635*, [Online]. Available: http://arxiv.org/abs/1905.06635

[30] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," 2016, *arXiv:1607.02533*. [Online]. Available: http://arxiv.org/abs/1607.02533

[31] Y. Dong, F. Liao, T. Pang, X. Hu, and J. Zhu, "Discovering adversarial examples with momentum," 2017, *arXiv:1710.06081*. [Online]. Available: http://arxiv.org/abs/1710.06081

[32] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017, *arXiv:1706.06083*.

[33] K. R. Mopuri, A. Ganeshan, and R. V. Babu, "Generalizable data-free objective for crafting universal adversarial perturbations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 10, pp. 2452–2465, Oct. 2019.

[34] A. Ilyas, L. Engstrom, and A. Madry, "Prior convictions: Black-box adversarial attacks with bandits and priors," 2019, *arXiv:1807.07978*.

[35] Z. Yan, Y. Guo, and C. Zhang, "Subspace attack: Exploiting promising subspaces for query-efficient black-box attacks," 2019, *arXiv:1906.04392*. [Online]. Available: http://arxiv.org/abs/1906.04392

[36] J. H. Metzen, M. C. Kumar, T. Brox, and V. Fischer, "Universal adversarial perturbations against semantic image segmentation," 2017, *arXiv:1704.05712*.

[37] C. Xie, J. Wang, Z. Zhang, Y. Zhou, L. Xie, and A. L. Yuille, "Adversarial examples for semantic segmentation and object detection," 2017, *arXiv:1703.08603*. [Online]. Available: http://arxiv.org/abs/1703.08603

[38] N. Papernot and P. D. McDaniel, "Extending defensive distillation," 2017, *arXiv:1705.05264*. [Online]. Available: http://arxiv.org/abs/1705.05264

[39] Y. Song, T. Kim, S. Nowozin, S. Ermon, and N. Kushman, "Pixeldefend: Leveraging generative models to understand and defend against adversarial examples," 2017, *arXiv:1710.10766*. [Online]. Available: http://arxiv.org/abs/1710.10766

[40] A. Athalye, N. Carlini, and D. A. Wagner, "Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples," 2018, *arXiv:1802.00420*. [Online]. Available: http://arxiv.org/abs/1802.00420

[41] D. Hendrycks and K. Gimpel, "Visible progress on adversarial images and a new saliency map," 2016, *arXiv:1608.00530*. [Online]. Available: http://arxiv.org/abs/1608.00530

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

[42] K. Grosse, P. Manoharan, N. Papernot, M. Backes, and P. D. McDaniel, "On the (statistical) detection of adversarial examples," 2017, *arXiv:1702.06280*. [Online]. Available: http://arxiv.org/abs/1702.06280

[43] R. Feinman, R. R. Curtin, S. Shintre, and A. B. Gardner, "Detecting adversarial samples from artifacts," 2017, *arXiv:1703.00410*.

[44] T. Miyato, S. Maeda, M. Koyama, and S. Ishii, "Virtual adversarial training: A regularization method for supervised and semi-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 8, pp. 1979–1993, Aug. 2019.

[45] A. Shafahi et al., "Adversarial training for free!," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2019, pp. 3358–3369. [Online]. Available: http://papers.nips.cc/paper/8597-adversarial-training-for-free.pdf

[46] H. Zhang, Y. Yu, J. Jiao, E. P. Xing, L. E. Ghaoui, and M. I. Jordan, "Theoretically principled trade-off between robustness and accuracy," 2019, *arXiv:1901.08573*. [Online]. Available: http://arxiv.org/abs/1901.08573

[47] T. Pang, K. Xu, C. Du, N. Chen, and J. Zhu, "Improving adversarial robustness via promoting ensemble diversity," 2019, *arXiv:1901.08846*. [Online]. Available: http://arxiv.org/abs/1901.08846

[48] M. Cisse, P. Bojanowski, E. Grave, Y. Dauphin, and N. Usunier, "Parseval networks: Improving robustness to adversarial examples," 2017, *arXiv:1704.08847*.

[49] T.-W. Weng et al., "Evaluating the robustness of neural networks: An extreme value theory approach," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–18. [Online]. Available: https://openreview.net/forum?id=BkUHlMZ0b

[50] A. Mustafa, S. H. Khan, M. Hayat, R. Goecke, J. Shen, and L. Shao, "Deeply supervised discriminative learning for adversarial defense," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 9, pp. 3154–3166, Sep. 2021.

[51] A. Brock, T. Lim, J. M. Ritchie, and N. Weston, "SMASH: One-shot model architecture search through hypernetworks," 2017, *arXiv:1708.05344*. [Online]. Available: http://arxiv.org/abs/1708.05344

[52] H. Pham, M. Y. Guan, B. Zoph, Q. V. Le, and J. Dean, "Efficient neural architecture search via parameter sharing," 2018, *arXiv:1802.03268*. [Online]. Available: http://arxiv.org/abs/1802.03268

[53] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," 2017, *arXiv:1707.07012*. [Online]. Available: http://arxiv.org/abs/1707.07012

[54] Z. Yang et al., "Cars: Continuous evolution for efficient neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, *arXiv:1909.04977*.

[55] X. Dong and Y. Yang, "NAS-bench-201: Extending the scope of reproducible neural architecture search," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–16. [Online]. Available: https://openreview.net/forum?id=HJxyZkBKDr

[56] H. Chen et al., "Anti-bandit neural architecture search for model defense," 2020, *arXiv:2008.00698*.

[57] D. V. Vargas and S. Kotyan, "Evolving robust neural architectures to defend from adversarial attacks," 2019, *arXiv:1906.11667*. [Online]. Available: http://arxiv.org/abs/1906.11667

[58] J. Mok, B. Na, H. Choe, and S. Yoon, "AdvRush: Searching for adversarially robust neural architectures," in *Proc. 2021 IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 12302–12312.

[59] R. Hosseini, X. Yang, and P. Xie, "DSRNA: Differentiable search of robust neural architectures," 2020, *arXiv:2012.06122*. [Online]. Available: https://arxiv.org/abs/2012.06122

[60] S. Jung, J. Lukasik, and M. Keuper, "Neural architecture design and robustness: A dataset," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 1–33. [Online]. Available: https://openreview.net/forum?id=p8coElqiSDw

[61] C. Devaguptapu, D. Agarwal, G. Mittal, P. Gopalani, and V. N. Balasubramanian, "On adversarial robustness: A neural architecture search perspective," in *Proc. 2021 IEEE/CVF Int. Conf. Comput. Vis. Workshops*, 2021, pp. 152–161.

[62] S. Huang, Z. Lu, K. Deb, and V. N. Boddeti, "Revisiting residual networks for adversarial robustness," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 8202–8211.

[63] A. Liu et al., "Exploring the relationship between architectural design and adversarially robust generalization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 4096–4107.

[64] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," 2018, *arXiv:1810.05270*. [Online]. Available: http://arxiv.org/abs/1810.05270

[65] C. Liu et al., "Progressive neural architecture search," 2017, *arXiv:1712.00559*. [Online]. Available: http://arxiv.org/abs/1712.00559

[66] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," 2018, *arXiv:1802.01548*. [Online]. Available: http://arxiv.org/abs/1802.01548

[67] Y. Yoshida and T. Miyato, "Spectral norm regularization for improving the generalizability of deep learning," 2017, *arXiv:1705.10941*.

[68] H. Qian and M. N. Wegman, "L2-nonexpansive neural networks," 2018, *arXiv:1802.07896*. [Online]. Available: http://arxiv.org/abs/1802.07896

[69] N. A. Marlow, "A normal limit theorem for power sums of independent random variables," *Bell Syst. Tech. J.*, vol. 46, no. 9, pp. 2081–2089, 1967.

[70] J. Zhang et al., "Attacks which do not kill training make adversarial learning stronger," 2020, *arXiv:2002.11242*. [Online]. Available: https://arxiv.org/abs/2002.11242

[71] Y. Wang, D. Zou, J. Yi, J. Bailey, X. Ma, and Q. Gu, "Improving adversarial robustness requires revisiting misclassified examples," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–14. [Online]. Available: https://openreview.net/forum?id=rklOg6EFwS

[72] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 39–57.

[73] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," 2020, *arXiv:2003.01690*. [Online]. Available: https://arxiv.org/abs/2003.01690

[74] N. Papernot, P. D. McDaniel, and I. J. Goodfellow, "Transferability in machine learning: From phenomena to black-box attacks using adversarial samples," 2016, *arXiv:1605.07277*. [Online]. Available: http://arxiv.org/abs/1605.07277

[75] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "Ensemble adversarial training: Attacks and defenses," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–20. [Online]. Available: https://openreview.net/forum?id=rkZvSe-RZ

Minjing Dong received the BS degree in software engineering from the Dalian University of Technology, the BS degree in information technology from the University of Sydney, the MPhil degree in engineering and information technology from the University of Sydney, and the PhD degree in engineering and information technology from the University of Sydney. He is currently an assistant professor with the department of computer science, City University of Hong Kong. His research interests include adversarial robustness, model calibration, efficient neural network, human motion analytics, and generative models. He has published more than 30 papers in top-tier conferences and journals, such as NeurIPS, ICML, CVPR, AAAI, ICLR, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Neural Networks and Learning Systems*, *IEEE Transactions on Image Processing*, and *IEEE Transactions on Multimedia*. He received AAAI 2023 Distinguished Paper Award. He is the area chair and PC member of many conferences and journals, such as NeurIPS, ICML, CVPR, ICCV, and ECCV.

Yanxi Li received the MPhil degree from the University of Sydney, in 2020 and subsequently began his PhD. He is currently working toward the PhD degree with the School of Computer Science, University of Sydney, Australia. His research focuses on neural architecture design and the robustness of deep vision models.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.

Yunhe Wang (Senior Member, IEEE) received the PhD degree from Peking University, China. He is currently a senior researcher with Huawei Noah's Ark Lab. His research interests mainly include machine learning, computer vision, and efficient deep learning. He has published more than 100 papers in prestigious journals and conferences. Many of them are widely applied in industrial products, and received important awards. He leads the "Adder Neural Networks" project, which presents a new computing paradigm for artificial intelligence with significantly lower hardware costs. He regularly serves as PC/senior PC member for top conferences, e.g., NeurIPS, ICML, ICLR, CVPR, ICCV, IJCAI, and AAAI.

Chang Xu (Senior Member, IEEE) received the PhD degree from Peking University, China. He is ARC future fellow and associate professor with the School of Computer Science, University of Sydney. He received the University of Sydney Vice-Chancellor's Award for Outstanding Early Career Research. His research interests lie in machine learning algorithms and applications in computer vision. He has published more than 100 papers in prestigious journals and top tier conferences. He has received several paper awards, including Distinguished Paper Award, in AAAI 2023 and Distinguished Paper Award in IJCAI 2018. He served as an area chair of NeurIPS, ICML, ICLR, KDD, CVPR, and MM, as well as a Senior PC member of AAAI and IJCAI. In addition, he served as an associate editor with *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Multimedia*, and *Transactions on Machine Learning Research*. He has been named a Top Ten Distinguished Senior PC Member in IJCAI 2017 and an Outstanding associate editor with *IEEE Transactions on Multimedia*, in 2022.

Authorized licensed use limited to: SHENZHEN UNIVERSITY. Downloaded on April 22,2025 at 08:25:25 UTC from IEEE Xplore. Restrictions apply.